

Apache Spark *and* Developing Applications Using Spark Streaming

Apache Spark is a unified analytics engine for Big Data processing, with built-in modules for streaming, SQL, machine learning and graph processing. This article covers various aspects of Spark and Spark Streaming.



“There’s so much buzz and press about Spark right now,” says Cloudera’s chief technologist, Eli Collins. The need for a powerful data processing engine arose when the data explosion took place in the IT world. Though we have many tools and technologies available to solve Big Data problems, they come with many pros and cons in terms of their processing capacity based on the data volumes, velocities and the sources where the data is created. Apache Spark provides a better solution than most. It can handle data at the peta scale and is capable of processing real-time streaming data. Besides, it can process data from different sources such as Twitter, Kafka, Cassandra, HDFS, S3, Kinesis, etc. Spark is much faster than the MapReduce engine because of its in-memory data processing capabilities. In this article, we will focus more on Apache Spark and its internals. Later, we will create a Spark streaming application to process real-time Twitter data.

All about Apache Spark

Apache Spark is an open source distributed general-purpose cluster computing framework with an in-memory data processing engine. With it, we can do Extract, Transform and Load (ETL) analysis, machine learning, and graph processing on large volumes of data at rest as well as data in motion, with rich, concise, high-level APIs in programming languages such as Python, Java, Scala, R and SQL. The Spark execution engine is capable of processing batch data and real-time

streaming data from different sources via Spark streaming. Spark supports an interactive shell for data exploration and for ad hoc analysis. We can run the Spark applications locally or distributed across a cluster, either by using an interactive shell or by submitting an application.

Spark uses three interactive shells.

spark-shell: The Scala interactive shell

pyspark: The Python interactive shell

sparkR: The R interactive shell

To run applications distributed across a cluster, Spark requires a cluster manager. The Spark environment can be deployed and managed through the following cluster managers.

- Spark Standalone
- Hadoop YARN
- Apache Mesos
- Kubernetes

Spark Standalone is the simplest tool to deploy Spark on a private cluster. Here, the Spark application processes are managed by Spark master and worker nodes. When Spark applications run on a YARN cluster manager, the former’s processes are managed by the YARN ResourceManager and NodeManager. YARN controls resource management, scheduling and the security of submitted applications.

Besides Spark on YARN, the Spark cluster can be managed by Mesos (a general cluster manager that can also run Hadoop MapReduce and service applications), as well as Kubernetes (an open source system for automating deployment,